

Tentamen

EDA482/DAT017/DAT390/DIT153 Maskinorienterad programmering

Lördag 14 mars 2026, kl. 08.30 - 12.30

Examinator

Erik Sintorn, tel. 031-772 52 29

Kontaktperson under tentamen:

Sergei Dytckov, tel. 0737085336

Erik Sintorn, tel. 031-772 52 29

Tillåtna hjälpmedel

Utgåvor som distribuerats inom ramen för kursen, häftet:

Quick Guide, Laborationsdator MD407 med tillbehör

Inget annat än understrykningar ("överstrykningar") får vara införda i detta häfte.

Tabellverk eller miniräknare får ej användas.

Uppgifter 1 t.o.m 2 utgörs av flervalsfrågor. Försök först lösa uppgifterna, så som du lärt dig i kursen, jämför sedan med flervalsalternativen och ange det alternativ du anser är korrekt. Högst ett svarsalternativ får anges. Siffror inom parentes anger möjliga poäng på uppgiften.

Observera att, bland svarsalternativen finns alternativ som kan ge poängavdrag. Om du är mycket osäker på ditt svar bör du därför i stället välja att avstå.

Avge dina flervalssvar på blanketten som följer med tentamenstesen. Skicka med denna blankett med dina övriga lösningar.

Tänk på:

- Att skriva tydligt och disponera papperet på ett lämpligt sätt.
- Att börja varje ny uppgift på nytt blad. Skriv endast på en sida av papperet.
- Att skriva den (anonyma) kod du fått på alla blad.
- De råd och anvisningar som givits under kursen skall följas vid programkonstruktionerna.

Följande riktlinjer gäller för utförlighet:

- Du kan svara på svenska eller engelska.
- Kommentera din kod.
- Om du bara kan svara på en del av uppgiften är det bra att beskriva i en kommentar hur kan man fortsätta.
- För funktioner vars deklaration ges i uppgiften behöver du ej upprepa funktionshuvudet, att skriva funktionskroppen under funktionens namn räcker.

Betygsättning Maximal poäng är 50 och tentamenspoäng ger slutbetyg enligt: $20p \leq \text{betyg } 3 < 30p \leq \text{betyg } 4 < 40p \leq \text{betyg } 5$

Uppgift 1 (-1.5 ... 9p)

a) Givet följande variabeldeklarationer i C kod:

```
char a;  
short b[5];  
int c;
```

Välj det svar som i assembler-kod allokerar minnesutrymme för variablerna och förhindrar alignmentfel. (-0.5...3p)

Svarsalternativ

A	B	C	D	E	F
<code>.align 2 a: .space 1 .align 1 b: .space 10 c: .space 4</code>	<code>a: .space 1 .align 1 b: .space 10 .align 1 c: .space 4</code>	<code>a: .space 1 .align 2 b: .space 10 .align 1 c: .space 4</code>	<code>.align 2 a: .space 1 b: .space 10 .align 2 c: .space 4</code>	<code>a: .space 1 b: .space 10 .align 1 c: .space 4</code>	<code>.align 1 a: .space 1 .align 2 b: .space 10 .align 1 c: .space 4</code>

b) Följande variabler är deklarerade i C kod:

```
int i;  
short arr[255];
```

och vi vill utföra följande tilldelningsoperation:

```
arr[i+1] = 0;
```

Vilken assembler-kod nedan implementerar det korrekt? (-0.5...3p)

Svarsalternativ

A	B	C
<code>LDR r0, i LDR r1, =arr LSL r0, r0, #1 ADD r0, r0, #1 ADD r0, r0, r1 MOV r2, #0 STRH r2, [r0]</code>	<code>LDR r0, i LDR r1, =arr ADD r0, r0, #1 LSL r0, r0, #2 ADD r0, r0, r1 MOV r2, #0 STRH r2, [r0]</code>	<code>LDR r0, i LDR r1, =arr ADD r0, r0, #2 LSL r1, r1, #1 ADD r0, r0, r1 MOV r2, #0 STRH r2, [r0]</code>
D	E	F
<code>LDR r0, i LDR r1, =arr ADD r0, r0, #1 LSL r0, r0, #1 ADD r0, r0, r1 MOV r2, #0 STRH r2, [r0]</code>	<code>LDR r0, i LDR r1, =arr LSL r0, r0, #2 ADD r0, r0, #1 ADD r0, r0, r1 MOV r2, #0 STRH r2, [r0]</code>	<code>LDR r0, i LDR r1, =arr ADD r0, r0, #2 LSL r0, r0, #1 ADD r0, r0, r1 MOV r2, #0 STRH r2, [r0]</code>

c) Följande variabler är deklarerade i C kod:

```
unsigned int k;  
signed char a[5];  
unsigned short s;
```

och vi vill utföra följande tilldelningsoperation:

```
a[0] = k - s;
```

Vilket svarsalternativ nedan implementerar det korrekt i assemblerkod? (-0.5...3p)

Svarsalternativ

A	B	C
LDR r0, k LDRSH r1, s SUB r2, r0, r1 LDR r3, =a STRB r2, [r3]	LDR r0, k LDRSH r1, s SUB r2, r1, r0 LDR r3, =a STRH r2, [r3]	LDR r0, k LDR r1, s SUB r2, r0, r1 LDR r3, =a STR r2, [r3]
D	E	F
LDRSB r0, k LDRH r1, s SUB r2, r0, r1 LDR r3, =a STR r2, [r3]	LDR r0, k LDRH r1, s SUB r2, r0, r1 LDR r3, =a STRB r2, [r3]	LDR r0, k LDRSH r1, s SUB r2, r0, r1 LDR r3, =a STRH r2, [r3]

Uppgift 2 (-2.5 ... 8p)

a) Givet följande C kod:

```
int current_mA;  
int load_mode;  
  
int estimate_power_mW(int current_mA);  
int calculate_factor(int load_mode, int base_power);
```

skall följande kod implementeras i assembler:

```
int compute_power_output(void) {  
    int base_power_mW = estimate_power_mW(current_mA);  
    int adjusted_power_mW = calculate_factor(load_mode, base_power_mW);  
    return adjusted_power_mW * base_power_mW;  
}
```

Välj det svarsalternativ nedan som implementerar `compute_power_output` korrekt i assembler (-1...3p).

Svarsalternativ

A	B	C	D
compute_power_output: PUSH {r4, lr} LDR r0, current_mA BL estimate_power_mW MOV r4, r0 LDR r1, load_mode MOV r0, r4 BL calculate_factor MUL r0, r0, r4 POP {r4, pc}	compute_power_output: LDR r0, current_mA BL estimate_power_mW STR r0, [sp] LDR r0, load_mode LDR r1, [sp] BL calculate_factor LDR r1, [sp] MUL r0, r0, r1 BX lr	compute_power_output: LDR r0, current_mA BL estimate_power_mW MOV r4, r0 LDR r0, load_mode MOV r1, r4 BL calculate_factor MUL r0, r0, r4 BX lr	compute_power_output: PUSH {r4, lr} LDR r0, current_mA BL estimate_power_mW MOV r4, r0 LDR r0, load_mode MOV r1, r4 BL calculate_factor MUL r0, r0, r4 POP {r4, pc}

b) Vi vill implementera apply_mask(int *array, int size, char mask)-funktionen som applicerar en mask på **fjärde** byten (mest signifikanta byten) av varje element i array.

Att applicera en mask är en enkel bitvis AND-operation.

Välj det svarsalternativ nedan som implementerar denna funktion korrekt (-1...3p).

Svarsalternativ

A	B
<pre>void apply_mask(int *array, int size, char mask) { for (int i = 0; i < size; i++, array++) { char *byte_ptr = (char *) (array + 3); *byte_ptr &= mask; } }</pre>	<pre>void apply_mask(int *array, int size, char mask) { for (int i = 0; i < size; i++) { char *byte_ptr = (char *) (array + i); byte_ptr[3] &= mask; } }</pre>
C	D
<pre>void apply_mask(int *array, int size, char mask) { for (int i = 0; i < size; i++) { char *byte_ptr = (char *) (array + i * sizeof(int)); byte_ptr[3] &= mask; } }</pre>	<pre>void apply_mask(int *array, int size, char mask) { char *byte_ptr = (char *) array; for (int i = 0; i < size; i++, byte_ptr++) { byte_ptr[3] &= mask; } }</pre>

c) Givet koden nedan:

```
void foo(int x, int *y) {
    x = x + 2;
    *y = *y + 3;
    y = &x;
    *y = 100;
}

int main() {
    int a = 5;
    int b = 5;

    foo(a, &b);

    printf("%d %d\n", a, b);
    return 0;
}
```

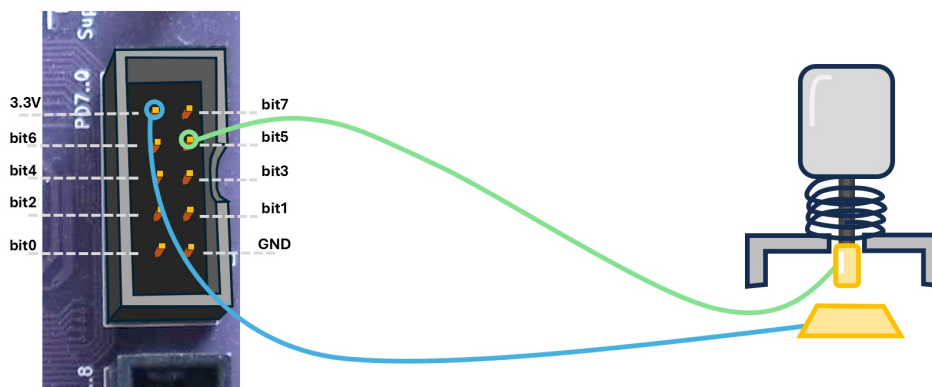
Vad kommer att skrivas ut? (-0.5...3p)

Svarsalternativ

A	B	C	D	E	F
5 5	7 8	100 8	5 100	5 8	100 5

Uppgift 3 (6p)

En knapp är kopplad till pin 5 i port D som visas på bilden nedan.



- Skriv makrodefinitioner för registren MODER, PUPDR, IDR och ODR i GPIO port D. Skriv dessutom en makrodefinition för ODR_LOW som endast ger tillgång till bitar [0..7] i ODR registret. (2p)
- Vilket GPIO-läge bör väljas för pin 5 i port D för att på ett tillförlitligt sätt läsa av knappens status? Motivera ditt svar i en eller två meningar. (Tips: Du kan rita en diagram om det hjälper.) (2p)
- Konfigurera pin 5 i GPIO port D i det läge du valde i del b). Konfigurationen av andra pins får inte ändras. (2p)

Uppgift 4 (7p)

Vi vill skriva en delay-rutin som kommer närmare en exakt delay genom att vi delar upp den i en konfigureringsfunktion och en delay funktion som sedan kan användas flera gånger.

Antag en systemklocka på 144MHz.

Ni får anta att makrodefinitioner för pekare till varje SysTick register (med namn enligt quickguiden) är givna.

- Skriv funktionen `void config_delay(int us)`, som konfigurerar SysTick för att räkna från 0 upp till det antal klockcykler som motsvarar `us` mikrosekunder. Denna funktion skall *inte* starta SysTick. (3p)
- Skriv funktionen `void delay()` som startar SysTick, väntar tills den nått det angivna värdet, och sedan stannar klockan. Funktionen skall gå att kalla flera gånger, med samma delay som resultat. (3p)
- Hur lång tid hade SysTick som mest kunnat räkna, om `CLKSOURCE` vore satt till noll, och `STK_VAL` bara vore ett 16-bitars register (1p)?

Uppgift 5 (6p)

Ett nytt peripheral har lagts till i MD407, med basadress 0xE0008000 och register layout som visas nedan. De gråmarkerade fälten får *inte* läsas eller skrivas.

Offset	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Register
0x0																																	CTRL
0x4																																	STATUS
0x8																																	
0xC																																	DATA

- Skriv en struct-definition som ger tillgång till de tre registren DATA, CTRL, och STATUS, och visa hur din struct kan användas för att skriva till DATA registret (32 bitar). (3p)
- Skriv en bitfäldsdeklaration för CTRL registret som ger tillgång till EN, MODE och SPEED fälten. (1.5p)
- Skriv en union som tillåter referens till DATA_LOW och DATA_HIGH fälten separat som 16-bitars värden, samt ger tillgång till hela 32-bitars registret DATA. (1.5p)

Uppgift 6 (7p)

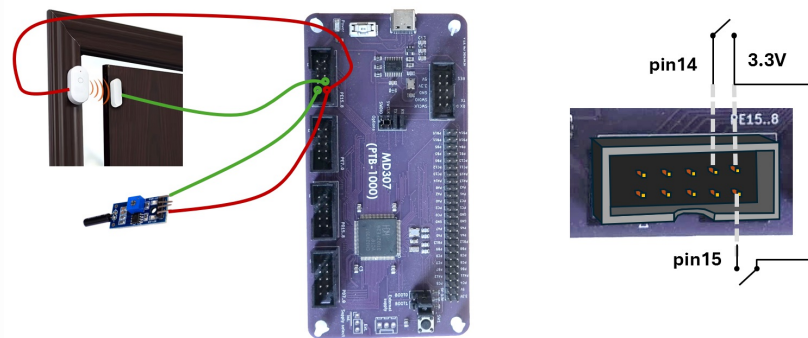
Två MD407-kort är anslutna via USART1-modulen. Ett kort samlar information från ljussensorer och skickar den till styrenheten som tar ett beslut om att stänga eller öppna persienner på en kontorsbyggnad. USART1-modulen är redan korrekt initialiserad och konfigurerad för att generera en interrupt när data tas emot (RXNEIE bit — RX Not Empty Interrupt Enable — i register USART1_CR1 är satt). En interrupthanterare, `usart_isr()`, är redan implementerad.

Din uppgift är att konfigurera styrenheten så att ett interrupt från USART1-modulen kommer att exekvera `usart_isr()` funktionen. Du kan utgå från att makrodefinitioner för pekare till alla nödvändiga register är givna, med namn enligt quickguiden.

- Konfigurera NVIC så att USART1 interrupt är aktiverat. (2p)
- Skriv C-kod som relokerar vektortabellen till adressen `0x2001C000` genom att sätta registret `SCB_VTOR`. (2p)
- Skriv C-kod som, efter relokeringen i b), placerar funktionspekaren till `usart_isr` på rätt plats i den relokerade vektortabellen (3p).

Uppgift 7 (7p)

Vi utvecklar ett hemlarmssystem. För att säkra dörrar har vi bestämt oss för att använda en magnetisk sensor som fungerar som en strömbrytare: när en dörr är stängd är strömbrytaren också stängd, när en dörr är öppen är strömbrytaren öppen. Dessutom är en vibrationssensor placerad på dörren för att förhindra att tjuvar skär ett hål i dörren utan att upptäckas. Elektriskt sett beter sig vibrationssensorn också som en strömbrytare: när rörelse sker stängs strömbrytaren. (Tänk dig en tråd upphängd med icke-ledande fjädrar i ett ledande hölje.)



Som ni ser på bilden ovan så är både den magnetiska sensorn och vibrationssensorn anslutna till GPIO port E. Den magnetiska sensorns utgång är ansluten till pin 14, och vibrationssensorns utgång är ansluten till pin 15. GPIO pinnar, NVIC och vektortabell är redan konfigurerade för att starta en interrupt handler när en extern interrupt sker på pin 14 eller 15 i port E. Antag att makrodefinitioner för pekare till alla nödvändiga register är givna, med namn enligt quickguiden. Antag också att MD407-kortet används för andra säkerhetskomponenter, och dina modifikationer får inte påverka dem.

- Skriv koden för att initiera EXTI och SYSCFG_EXTICR så att interrupt genereras när den magnetiska sensorn eller vibrationssensorn detekterar intrång (3p)
- Färdigställ interrupt handlern nedan så att den, vid detekterat intrång, anropar funktionen `sound_alarm(int source)` med rätt source-värde (1 för magnetisk sensor, 2 för vibrationssensor). (4p)

```
void interrupt_handler(void) {  
    <Din kod här>  
}
```

Svarsblankett

Anonym kod: _____

Maskinorienterad programmering, tentamen flervalsfrågor. Kryssa i det alternativ du anser vara korrekt, eller avstå. Lämna in denna blankett tillsammans med dina övriga lösningar

	A	B	C	D	E	F
Uppgift 1a						
Uppgift 1b						
Uppgift 1c						
Uppgift 2a						
Uppgift 2b						
Uppgift 2c						